

AgentIR Technical Report — Tools v0.2

one surface, vLLM’s own tool machinery underneath: the integration contract, the hazard audit, and the amendments it forced

Yuxuan Sun, Claude Fable 5

tools-v0.2.2 (implementation-aligned specification) · July 2026

Summary. Tools-v0.1 [1] established the kernel objects for tool calls in AgentIR — T-nodes (tool executions with their own identity), the Obs spine segment (late-binding observation bytes), environment-resolved decisions (selections resolved by parser events rather than program values), and two-class-plus-modifier effects. This report revises the *surface* and the *realization*: the user-facing primitive collapses to one call, `infer(x, tools=..., max_turns=...)`, whose lowering reuses vLLM’s own tool machinery — chat-template rendering, the structural-tag registry, the tool-parser stack, and the engine-side stop merge — below the serving layer, instead of AgentIR-owned grammar and projection machinery. A six-agent source-level hazard audit of the pinned tree (vLLM 9fde043f5, AgentIR 8a63540; one agent executed the parser stack in-process) found six blockers and some twenty majors in the naive version of that plan; every amendment in this report traces to one of them, and the audit also surfaced one large positive: vLLM ships a token-in/token-out HTTP tier that upgrades the degrade path. Kernel semantics from v0.1 are unchanged; where this report says nothing, v0.1 stands. Revision v0.2.1 adds the per-tool *observation contract* (§2) — the declaration that tells the runtime how a continuation may run ahead of a tool’s result — absorbs the externally validated pieces of the Speculative Actions framework [3], and scopes the specification to fully autonomous runs (no human in the loop). Revision v0.2.2 promotes *predictors to first-class citizens*: a `guess` may be an arbitrary effect-free sub-program — in particular a draft-model generation — whose output is the hit slot’s bytes, with a legality rule, an asymmetric failure semantics, and a scheduling account (§2).

Implementation-status convention. This revision describes the shipped vLLM integration, while retaining the original hazard audit as design rationale. Statements in §4 about what the “current” spine or parser could not do refer to the audited pre-implementation baseline; the amendments are implemented unless an explicit limitation is stated. The supported execution modes are Tier 0 and Tier 1. Tiers 2–4 in §5 are a future capability sketch, not implemented fallback paths.

1 The Surface: One Primitive, Three Layers

The user-facing addition is a single argument to the existing primitive:

```
tools = {
  "search": ToolDef(fn=web_search, effect="speculable",
    obs=Project(top_snippet), guess=memo.lookup),
  "tests": ToolDef(fn=run_tests, effect="unknown",
    obs=Enum({"pass": "0 failed", "fail": open()})),
  "log": ToolDef(fn=append_log, effect="unknown", obs=Ignore("done")),
}
y = infer(x, tools=tools, max_turns=8) # returns the final Seq
```

`effect=` governs when the tool *body* may run (unchanged from v0.1); `obs=` and `guess=` form the observation contract of §2 and govern when the *continuation* may run. The two axes are orthogonal.

The model decides at runtime whether, when, which, and how many tools to call; the program declares only the binding (name → function, effect class) and the budget. There is no user-visible parsing, dispatch, or loop. The v0.1 primitives (`calls`, `tool`, `obs`, `expect`, `classify`) remain as the *advanced layer* for programs that control speculation manually — the relation is that of `choose` to the hoist: the general form is what almost everyone writes; the explicit forms are what it lowers through.

Lowering: topology is pre-registered; physical continuations wait for a call prefix. “No-body knows when the model will emit a tool call” does not require runtime-generated identity. Before the parser reveals a call, the tracer reserves a *fixed round skeleton*: one continuation draw, a family of K deferred call slots, K publication gates, and $K+1$ existence gates (the extra gate is the overflow sentinel). This pre-registers topology and draw numbering, not a runnable continuation E-node: its prompt still needs the model-emitted tool name, arguments, and call id. As soon as the prefix-stable parser closes a call’s arguments (an event that is never revised), the corresponding slot binds and the tracer registers the viable round- $t+1$ continuation worlds against the reserved draw. Terminal parsing then validates exactly one call-inventory prefix and squashes the others; a no-call turn closes the skeleton without ever materializing a physical continuation. Thus the skeleton exists before anyone knows whether the model will call a tool, while physical continuation E-nodes register only after prefix-stable call closure. Per-round call multiplicity remains bounded by K ; overflow degrades to a terminal barrier — cost, never soundness.

Scope: fully autonomous runs. This specification assumes no human participates during a run. Two consequences follow. First, buffering speculative output until commit costs nothing — there is no observer mid-run — so the streaming-versus-speculation conflict that interactive deployments must manage does not arise here. Second, the flagship long-latency tool in an all-agent system is the *sub-agent call* (a delegated multi-turn run whose returned summary is the observation): it carries the largest latency in the system and its summary is often predictable, making it the primary target of the observation contract below. Human-facing patterns (an approval prompt as a two-class codomain) remain expressible but are out of scope.

2 The Observation Contract

The `obs=` field declares, per tool, the partition of the observation that the program is allowed to depend on — v0.1’s three partitions surfacing as `ToolDef` data. The runtime derives the speculation mode from the declaration; the kernel object underneath is in every case the same environment-resolved decision over a declared partition of the projection image.

<code>obs=</code>	Meaning	Runtime behavior / waste
<code>Ignore(text)</code>	output does not matter; a declared constant stands in as the observation	continuation fully determined at call time — no wait, no verification, <i>deterministic</i> rather than speculative; real output recorded, never injected; zero waste
<code>Enum(cases, by)</code>	the projection’s codomain is a declared finite set; closed classes carry representative bytes; classes may be <code>open()</code>	a single-call hoist decodes all closed-class continuations concurrently; the real result selects by byte equality on the representative; open classes are Obs-hole branches; waste = losing branches’ decode
<code>Project(alpha) + guess</code>	only $\alpha(\text{result})$ enters the prompt; <code>guess</code> supplies candidate images	predict-and-verify: k hit slots plus OTHER; verification is byte equality on the α -image; waste = missed candidates’ decode
<code>Exact()</code> (default)	the full observation enters the prompt	Obs hole; no speculation unless <code>guess</code> is supplied (then as above with $\alpha = \text{id}$)

Table 1: The observation contract. Narrowing the projection is the hit-rate amplifier: a continuation that reads only an exit code should declare α as the exit code, not the transcript.

Five rules carry over from v0.1 unchanged and bind here: matching is byte equality on representatives (a class *is* its bytes; coarser equivalence is expressed by coarsening α); `Ignore`’s constant is trace-time data, enters identity, and is injected identically by the serial reference, so bit-identity is trivial — a tool failure under `Ignore` is unobservable unless the program gates on `ack`, matching unread-op semantics; injected bytes pass through the per-family suffix specification of §4 (A5/A7) — template-conformant wrapping is the runtime’s job, never the user’s; α is a named, pure declaration evaluated once per resolution with its image recorded; and predictors propose, never substitute.

First-class predictors (v0.2.2). `guess` generalizes from a value to a *predictor sub-program*: a

traced function `(state, call) -> Seq | [Seq]` built from DSL primitives, invoked by the runtime inside the hit world’s scope. The three simple sources become special cases — a string is `lit(...)`; a memo lookup is `op(pure)` then embedding; and the general case is a draft-model generation: `expand(cat(PRED_TMPL, recent(state), args), k=2, model="draft", max_tokens=32)`. The hit slot’s representative thereby generalizes from literal bytes to a `Seq` whose spine may contain `Out(epred)`. Once prefix-stable call closure has declared that slot, the hit continuation registers symbolically without waiting for predictor output, becomes data-ready when the predictor finishes, and decodes speculatively under the pending match gate. Soundness is inherited from seed-by-identity: the predictor’s bytes are reproducible, so the serial reference re-derives the same guess, performs the same comparison, and takes the same branch — predictor draws appear in both arms of the differential, which is also why a predictor must be first-class rather than hidden inside an `op` (the v1.2 precedent: the reward scorer rewritten as a judge-role engine call). Four rules complete the promotion:

- **Legality.** A predictor sub-program lives permanently in a speculative world, so every node in it must be speculation-eligible: `infer/expand`, `op(pure)`, and `speculable` T-nodes (a cheap simulator tool is thereby admitted); `unknown` effects are forbidden.
- **Asymmetric failure.** A predictor that fails or times out resolves the match decision as a graceful miss (zero hit slots; the Obs-hole path proceeds) — advisory compute may never fail the program. The real-observation side is unchanged: a projection landing outside a closed partition still fails the decision with cause.
- **Scheduling.** Predictor nodes are funded from the speculation budget and are preemptible by committed work; the predictor’s own output logprobs may serve as advisory hints; a draft model co-hosts on reserved engine memory via the existing multi-engine mechanism. The win window is tool latency minus predictor latency, and the predictor may start as soon as the call’s arguments close mid-decode.
- **Resolution ties.** If several slots byte-match the α -image, the first in declaration (or lane) order wins — stable order, never completion time.

A generic tool with no declared structure (an arbitrary `bash`) defaults to `guess=predict(model="draft", k=2)` — the universal fallback that the specialized contracts refine. One expressibility theorem is worth recording: single-turn predict-and-verify with a model predictor is already expressible in the v1.2 kernel (predictor as `infer`, match as a pure ranking, branches as `gates`; the miss branch embeds the real observation at a tracer wait) — what it lacks is exactly this report’s inventory: early dispatch, cross-round pipelining over Obs, recorded observations, and the declarative surface. That theorem licensed the historical P0.5 de-risking demo (deterministic tools, measured hit rates) before the T-node implementation landed.

k -way guesses and external calibration. `guess` may supply several candidates; with per-candidate accuracy p the probability that some slot hits is $p(k) = 1 - (1 - p)^k$. The Speculative Actions framework [3] validates this regime empirically — next-action prediction accuracy rising from 31% ($k=1$) to 55% ($k=3$) in a turn-based environment, for $\sim 20\%$ end-to-end savings from single-step speculation — and derives closed-form expected savings plus the cost-optimal k as a function of p , which the scheduler adopts for its default branch count. Two further pieces are adopted: the same-model, low-reasoning-budget speculator as the default `guess` predictor recipe; and one positioning fact stated plainly — their commit rule (reuse a pre-launched call when the guessed response induces the same next call specifier, held in a pending-call cache keyed by that specifier) is, in this kernel’s vocabulary, byte equality on the induced next node’s content-addressed identity: the cache is an ad-hoc `NodeId`. Their single-step scheme bounds end-to-end savings at 50%; the round-template unrolling of §1 is not subject to that bound.

Current multi-call bound. Per-call k -way slots are implemented, including stable-first selection and the real-observation hole. For a round containing several calls, however, the current vLLM lowering deliberately avoids the Cartesian product: it chooses the first declared closed slot for each call and registers one conjunction-guarded, declaration-first top-1 observation vector. This is a bounded heuristic, not a full Cartesian k -way decoder and not a logprob-ranked top-hint policy. General calibrated multi-vector selection remains future work.

3 The Integration Contract: vLLM’s Machinery, Reused

Tier 0 runs the engine in-process and therefore reuses the serving stack’s tool components directly; Tier 1 delegates the corresponding operations to the pinned vLLM render/TITO/derender endpoints. Table 2 is the normative mapping; everything format-specific remains vLLM code.

Role	vLLM component (pinned tree)	AgentIR’s part
tools into the prompt	the model’s chat template: <code>apply_chat_template(msgs, tools=...)</code> — rendered once per logical E-prefix; harmony (gpt-oss) via <code>openai_harmony</code> instead (§4, A5)	render at trace time into the spine; re-render at every tool-result/new-user boundary, then grow token-exactly within that registered generation
constrained decoding	<code>structural_tag_registry</code> . <code>get_model_structural_tag(model, tools, tool_choice, reasoning)</code> → <code>StructuredOutputsParams</code>	an <i>optional</i> capability (§4 A2); canonical tag string enters <code>NodeId</code>
turn boundary	<code>generation_config</code> eos list, merged engine-side (<code>input_processor.py:323</code>) — free in-process; the serving-layer stop merge is dead code at this commit	<code>stop_reason</code> surfaced; Token-Wire mandatory (§4 A7)
projection (calls out)	the model’s registered parser via <code>DelegatingParser.parse_delta</code> , followed by a fresh terminal <code>DelegatingParser.parse</code> cross-check	one incremental parser per physical attempt plus a fresh full parser at terminal; events are published on the record and replay-verified; a compat shim pins both entry points
tool-result re-entry	per-family suffix specification (§4, A5/A7) with template-faithful escaping	literal parts as Lits, result bytes as <code>Obs(t)</code> segments

Table 2: The reuse contract. AgentIR owns identity, guards, speculation, scheduling, and the check gate; vLLM owns every format-specific fact. Empirically verified: the parser stack imports and runs without any server (no `fastapi`/`uvicorn` loaded); a fabricated `ChatCompletionRequest` costs 0.2 ms; legacy-parser vocab instantiation costs 27 ms and must be memoized.

Engine configuration checklist (each item audit-verified, with the failure it prevents): structured-output backend `auto` or `xgrammar` *only* — guidance understands only the legacy tag format (`KeyError` escaping the fallback chain) and outlines passes validation then fails inside the engine core; `StructuredOutputsConfig(reasoning_parser=...)` set for reasoning models, else the grammar bitmask applies from token 0; requests enter only via `AsyncLLM.add_request` (the backend field is set by the input processor); `skip_special_tokens=False` per node (a serving-side parser mutation AgentIR would otherwise bypass — special-token call markers vanish from text without it); stop *token ids* only, never stop strings (strings truncate text but not token ids); never `min_tokens` with a grammar (EOS accepted → matcher terminates → suppressed stop → next token rejected → request error); with a drafter configured, no `min_p/logit_bias`, and note drafter on/off changes the seeded RNG path, so committed traces are comparable only within one drafter configuration.

4 The Hazard Audit and Its Amendments

Six audit lenses (parser coupling; template reality; engine configuration; the HTTP path; an adversarial sweep; AgentIR-side fit) produced six blockers and ~20 majors. The blockers and the amendments they forced are recorded below in their original causal order. The failure descriptions refer to the audited baseline; each “Amendment” sentence describes the current Tier 0 implementation unless a limitation is stated explicitly.

A1 — The Obs segment moves to the critical path. The audited spine admitted only Lit and Out; a tool result was neither, and embedding a control-plane value blocked the tracer. Without an amendment, round $t+1$ could not become data-ready until round t ’s tool returned. Amendment (implemented): `Obs(t, p)` is a first-class segment in both interpreters, naming a T-node and observation projection. It lets a continuation register after call closure while its observation bytes remain late-bound,

enabling cross-round overlap.

A2 — The grammar is an optional capability, not a premise. Under the default agent configuration (`tool_choice="auto"`, tools without `strict:true`) the registry returns *no grammar at all* — three stacked gates (`structural_tag_model` set on only 11 of 43 parser families; the `VLLM_ENFORCE_STRICT_TOOL_CALLING` env, read at import time in one place and call time in another, hence process-global; the per-tool strict flag). Decoding is unconstrained and the parser is the only projection. Forcing a grammar changes the sampling distribution and invalidates A/B comparison. Amendment: partition guarantees (schema-valid-by-construction, closed call sets) hold *only when the tag exists*; otherwise parse failure is a first-class outcome (decision fails with cause), and the speculation gates must not assume grammar-closed codomains.

A3 — The token-exact spine poisons the reasoning gate. Keeping previous turns verbatim leaves their `</think>` markers on the prompt; the engine’s reasoning-end detector scans backward and declares reasoning over, so from turn 2 the grammar bitmask applies from token 0 and a model that opens a fresh think block is killed by grammar rejection. Amendment: pass `reasoning_ended=False` explicitly per assistant turn (`AsyncLLM.add_request` exposes it) and configure the engine-side reasoning parser; mirror the flag into the in-process parser state.

A4 — Parsers are single-use; retries must re-verify. Empirically confirmed: parser instances have no reset API, and re-feeding a used instance corrupts indices and offsets. Amendment: one fresh parser per physical attempt for `parse_delta`, plus a separate fresh parser for the terminal `parse` cross-check (the serving layer’s own single-use pattern); events already published from a dead attempt are checked as a prefix against the replay — seed determinism makes byte-identical replay the expected case — and a mismatch poisons the node (a published divergence is not retryable, because decision resolution is exactly-once).

A5 — Family dispatch is structural. `gpt-oss/harmony` never touches the chat template (token ids rendered by `openai_harmony`; tools live in a developer message; tool results need a name, not an id). Llama-3.x JSON has *no* prefix-stable has-call gate (the parser’s trigger is a `startswith("{")` heuristic), so that family’s gate resolves at turn end only. Amendment: a per-family capability record — template path, early-gate availability, suffix specification — is first-class data, not special cases in code.

A6 — A bad tag kills the whole engine. Structural-tag compilation runs on an engine thread pool; an exception surfaces inside the scheduler loop and takes down every in-flight request (`EngineDeadError`; vLLM’s own TODO acknowledges the gap). In-process, that is all concurrent speculative attempts. Amendment: every distinct tag is validated *and* dry-run compiled (cached) before any `add_request`; engine death is a global scheduler event (commit nothing, restart). Tier 1 exposes no safe remote compile endpoint in the pinned protocol, so a returned structural tag fails closed before TITO `generate` rather than claiming Tier 0’s preflight guarantee.

A7 — The spine guarantee is scoped to one registered generation. The template audit shows none of the four major families is append-only across turns: Qwen retracts an already-rendered `<|im_end|>` when consecutive tool results merge; every family strips or rewrites prior-turn reasoning at round boundaries (while keeping it *within* a round — matching training distribution); Llama re-wraps tool results in a JSON envelope and rewrites the model’s emitted terminal token; DeepSeek-V3.1 renders no assistant header after tool results. Amendment: the token-exact spine guarantee holds within one registered generation. Every tool-result re-entry and every new user turn is a mandatory template re-render barrier. For a speculative observation, the renderer first sees a unique sentinel; the lowering replaces that one rendered span with the candidate or Obs fragment, preserving the family-owned prefix and suffix. Two collateral rules: TokenWire is mandatory for tool loops (the boundary token is kept in token ids but excluded from text, so a text spine is silently malformed), and Llama’s template date (`strftime_now`) must be pinned.

Selected majors with teeth (full catalog in the audit records): the `required-mode` streaming helper misattributes argument bytes across call boundaries inside one coalesced delta — never used as the projection; `parallel_tool_calls=False` is a wire-level post-filter, so a spine that keeps the dropped calls and a wire that hides them diverge — either enforce single-call at the grammar or do not offer the flag; parser regex timeouts (1s wall clock) make the projection load-dependent — guarded so a timeout is a node error rather than an ordinary-text downgrade; tool-call ids are random per response and two families render ids into the prompt — replaced by seed-derived deterministic ids; the parsed-but-unstreamed argument tail must be flushed explicitly at turn end or streaming and non-streaming projections of the same spine disagree.

5 Implemented Remote Modes and Future Degrade Sketch

The audit’s one large positive: the pinned tree ships a token-in/token-out HTTP tier (`scale_out`, registered on a normal `vllm serve`) — `/v1/chat/completions/render` performs template rendering plus `adjust_request` (hence the grammar) server-side and returns the prompt token ids and adjusted sampling parameters *as data*; `/inference/v1/generate` is raw token-in/token-out; `derender` runs the server’s parser without a GPU. AgentIR currently implements the first two modes below: Tier 0 in-process and Tier 1 render/TITO/derender. Rows 2–4 record protocol capabilities found by the audit and the semantics a future backend could retain; there is no Tier 2, Tier 3, or Tier 4 backend, automatic selection, or fallback path in tools-v0.2.2.

Tier	Path	What survives
0 (implemented)	in-process <code>AsyncLLM</code>	everything: mid-decode events, engine signals, per-node seeds, bit-identity gate
1 (implemented)	render + TITO + derender endpoints	token-exact spine and the server’s own grammar/parser stack; no mid-decode engine signals; seed honored, bit-identity subject to server batch invariance
2 (future sketch)	raw <code>/v1/completions</code> (client renders, client grammar, client parser over SSE)	potential token-exact spine; endpoint audit found structural tags, token-id prompts, seed, logprobs, and per-chunk token ids; traps: <code>add_special_tokens</code> default double-BOS, no <code>reasoning_ended</code> signal
3 (future sketch)	chat + <code>return_token_ids</code>	observation-complete, control-impossible: raw token stream recoverable for a <i>shadow spine</i> audit, but every turn is re-rendered server-side
4 (future sketch)	plain OpenAI-compatible chat	message-level only; no spine, no bit-identity; capability-degrade semantics of v1.2 apply

Table 3: Implemented modes and future capability sketch. Tier 1 is the only implemented remote mode; the render endpoint also serves as its server-authoritative prompt oracle. Tiers 2–4 are not AgentIR backends.

6 What Changed vs. v0.1, and Current Status

	tools-v0.1	tools-v0.2
user surface	<code>grammar=G, calls(), tool(), obs(), expect()</code>	<code>infer(x, tools=, max_turns=);</code> v0.1 forms demoted to the advanced layer
grammar/projection	AgentIR-declared combinators; content-addressed declarative α	vLLM’s registry and parsers reused; projection identity = (parser name, vLLM version pin)
loop structure	user-traced	fixed family/slot skeleton and continuation draw reserved first; physical E-nodes after prefix-stable call closure
Obs segment	P2	P1 (A1)
grammar status	assumed	optional capability (A2)
spine guarantee	global token-exact	per registered generation; tool-result and user turns are re-render barriers (A7)
HTTP fallback	binary degrade	Tier 0/Tier 1 implemented; Tiers 2–4 are future sketches (§5)
speculation enablers	expect only	per-tool obs= contract; guess = first-class predictor sub-program incl. draft-model generation and per-call k -way; multi-call lowering uses one declaration-first top-1 vector (§2)
scope	interactive allowed	fully autonomous runs; sub-agent-as-tool flagship

Table 4: Delta and implementation boundary. Kernel objects (T-node identity, effect classes, environment-resolved decisions, byte-equality matching, record/replay gate) are unchanged from v0.1.

Implementation status. The historical P0–P3 sequence has landed for the supported vLLM modes: canonical parameter validation; incremental event publication; Obs in guarded and serial interpreters; fresh incremental parsers with retry-prefix verification and fresh terminal cross-checks; family capability records; Tier 0 structural-tag compilation preflight and engine handshake; public `infer(tools=)`; observation contracts and first-class predictors; Tier 1 render/TITO/derender; Harmony; the nested sub-agent-as-tool demonstration; and preregistered paired benchmarks. “Landed” here does not erase the explicit bounds above: pre-call work is a skeleton rather than a physical continuation, multi-call speculation is one declaration-first top-1 vector, and the only implemented remote mode is Tier 1. Full calibrated multi-vector selection and remote Tiers 2–4 remain future work.

Two risks are named rather than hidden: the parser stack is mid-migration upstream (22 commits in 30 days across the two parser directories) — integration is confined to `DelegatingParser.parse_delta` plus the terminal `DelegatingParser.parse` cross-check behind a shim, with the tree pinned; and Tier 1 bit-identity is conditional on server batch invariance, exactly as the v1.2 certified-configuration rule already states.

References

- [1] Y. Sun, Claude. AgentIR Technical Report — Tools (tools-v0.1): typed outputs on the call side, environment-resolved decisions on the result side. Project document, July 2026.
- [2] Y. Sun, Claude. AgentIR Technical Report v1.2 — the guarded IR, minimized. Project document, July 2026.
- [3] N. Ye, A. Ahuja, G. Liargkovas, Y. Lu, K. Kaffes, T. Peng. Speculative Actions: A Lossless Framework for Faster Agentic Systems. arXiv:2510.04371.